Server is always listening to a port.
Server is called so, bcz it serves a software.
Server is a infinitely running software.

The diff btw HTTP and HTTPS is encryption certificate.
Request → sent by user
Response → sent by server

⇒ Module is used most prominently, thus we use "import" syntax

⇒ Making of a raw server :

```
const http = require('http');
const server = http.createServer((req, res) => {
    if (req.method === 'GET' && req.url === '/menu'){
        res.writeHead(200, {'Content-Type': 'application/json'})
        res.end(JSON.stringify({'thali', 'biryani'}))
    }
```

menu route

Request and response needs to be studied in NodeJS documentation eg flushHeders, etc. Request has its own event listners.

res.end is specified to send the response

<u>continued code</u> ⇒

```
} else if{ req.method ==="POST" && req.url ===  '/order'){
    let data = ''
    req.on('data', chunk => data += chunk)
    req.on('end', () => {
    const order = JSONparse(data)
res.writeHead(200, {"Content-Type": "application/
json"});
```

```
res.end( JSON.stringify({
        Status : 'received',
        order
    }));
  });
});
```

Data in above code is accepted on an event listener, the data is added in chunks. The 'on' is the event listner. And after .end is done, we use JSON.stringify to update.

Dependency Tree : when you install Express, you also install its dependencies, and those dependencies have their own dependencies.

Package-lock.json keeps the exact version of all the dependencies.

⇒
```
const express = require('express')
const app = express()
app.use(express.json())
app.get("/menu", (req, res) => res.json({
        items : ['thali', 'biryani']
}))
app.post("/order", (req, res) => {
    res.status(200).json({
        status : "recieved",
        order : req.body
    });
})
```

does all the chunking stuff done in previous code.

MVC (Model View Controller) Architecture.

(src), source code folder has all code of the application,

We can't run TS files directly.

tsconfig.json ⇒ tsc -init, In this we need to change two things.

"rootDir": "./src", → saves at TS files

"outDir": "./dist",

↑

Saves the JS files after compilation

tsc -p . ⇒ The TS compiler converts .ts to .js.

-p ⇒ It asks the compiler to use the configuration in tsconfig.json.

. ⇒ signifies current directory.

npx gitignore node → make .gitignore files for Node JS project

To automatically compile the code : Nodemon can be used.

Always prefered to install typescript both globally and also in the project.

Dev-dependencies : It installs packages as development dependency and not production dependency.

→ You can also do this - In package.json

```
"scripts": {
  "dev": "tsc-watch --onSucess \"node dist/index.js\"
},
```

Cmd: "npm i -D typescript tsc-watch"

npm i @types/<package-name> -D as without this express will not have it types.

4

npm i @types/node -D to state types for all inbuilt package in NodeJS

⇒

```
const PORT :number = process.env.PORT ? +process.env.PORT : 8080
```

typecast to make sure, us cannot add string or etc.

Zod (is a validation library)

⇓

```
import {z} from "zod";

const envSchema = z.object({
    PORT : z.string().optional()
})
```

Environment variables are mostly strings, you can always typecast it.

↳ to replace this use: import {env} from './env.js'

```
const PORT : number = env.PORT ? +env.PORT : 8080
```

⇒ `const server = http.createServer(createServerApplication());`

you delegate your things to app.js

For this to work use:

```
import {createServerApplication} from "./app/index.js"
```

Tip : Make a .vscode folder, where you can write custom code snippet.

< Schema.ts > file used to define the structure of data in TS. describes what fields exist, their types and some validation rules.

HW : Dry run TS code ;
Study TS whole week